



Tutor Académico Personal con Agentes LLM

Institución: Universidad Tecnológica Nacional Facultad Regional Santa Fe

Materia: Inteligencia Artificial

Curso: 5to Año - Ingeniería en Sistemas de Información

Profesores: Gutierrez María De Los Milagros, Roa Jorge

Integrantes:

- Bernard, Maximiliano - bmaximiliano0210@gmail.com
- Brunas, Kevin - kevinbrunas@hotmail.com
- Paez, Santiago santiagojosepaez2003@gmail.com
- Paggi, Santino - paggisantino@gmail.com

Año: 2026

1. Resumen Ejecutivo

Este documento define los requerimientos, arquitectura y criterios de aceptación del Tutor Académico Personal, un sistema de agentes inteligentes basados en Modelos de Lenguaje (LLMs) diseñado para asistir a estudiantes universitarios en la preparación de exámenes. El sistema combina técnicas de Retrieval-Augmented Generation (RAG), memoria persistente entre sesiones, herramientas (Tools) especializadas y una arquitectura multi-agente con roles diferenciados.

El agente ingiere material académico (apuntes de cátedra, exámenes anteriores), lo estructura en un índice temático consultable y genera artefactos de estudio personalizados: exámenes de opción múltiple, preguntas de respuesta libre y ejercicios prácticos. Un agente evaluador corrige las respuestas del estudiante, produce feedback detallado e identifica áreas de debilidad para adaptar el plan de estudio en sesiones futuras.

Propuesta de Valor

Reemplazar el estudio pasivo por un ciclo activo de generación, resolución, corrección, retroalimentación, personalizado al material propio de la cátedra y al historial de desempeño del estudiante.

2. Objetivo y Alcance

2.1 Objetivo del Sistema

Construir un agente inteligente autónomo capaz de: (a) ingerir y estructurar material académico en una base de conocimiento consultable; (b) generar artefactos de estudio (exámenes, cuestionarios, flashcards, ejercicios) adaptados a los contenidos y al perfil de desempeño del usuario; (c) evaluar respuestas del estudiante y proveer feedback accionable; (d) mantener un modelo del progreso del usuario entre sesiones para adaptar el foco de estudio.

2.2 Alcance Funcional

Incluido en el Alcance	Fuera del Alcance
Ingesta de PDFs, imágenes y texto plano	Generación de resúmenes automáticos de libros
Generación de exámenes y ejercicios	Corrección gramatical / estilo de redacción
Evaluación de respuestas libres con LLM	Integración con plataformas LMS (Moodle, etc.)
OCR y parsing de expresiones matemáticas	Generación de contenido de libros externos
Memoria persistente entre sesiones	Soporte multiusuario / multi-cátedra simultáneo
Panel de progreso y estadísticas	Modo offline sin API de LLM
Preferencias de estilo de examen por usuario	Certificación formal o acreditación académica

3. Ambiente del Agente

El ambiente en que el agente se desenvuelve es parcialmente observable. A continuación se caracteriza según la taxonomía de Russell y Norvig:

Dimensión	Clasificación	Justificación
Observabilidad	Parcialmente observable	El agente conoce el material ingestado y el historial, pero no el estado cognitivo real del estudiante
Agentes	Multi-agente (cooperativo)	Ingestor, ExamGenerator, ExerciseGenerator, Evaluator, SupportAgent coordinados
Determinismo	Estocástico	Las respuestas del LLM son no deterministas; el estudiante puede responder de formas imprevistas
Episodicidad	Secuencial	Cada sesión es independiente pero el estado del estudiante persiste entre sesiones
Dinamismo	Dinámico	El conocimiento del usuario evoluciona; el material puede actualizarse durante el curso
Continuidad	Contínuo	Se procesa lenguaje natural, por lo que el conjunto de entrada es prácticamente infinito.

3.1 Percepciones

Percepción	Descripción
Archivos subidos	PDFs, imágenes (fotos de apuntes), texto plano de apuntes y exámenes previos
Respuestas del estudiante	Texto libre o selección de opción múltiple como respuesta a preguntas generadas
Imágenes de exámenes resueltos	Fotos de hojas con ejercicios resueltos a mano, incluyendo fórmulas matemáticas
Preferencias del usuario	Configuración: tipo de preguntas preferidas, temas a priorizar, nivel de dificultad
Historial de sesiones	Puntajes, temas consultados y áreas con bajo desempeño de sesiones anteriores
Mensajes de chat	Instrucciones en lenguaje natural del estudiante para guiar la generación

3.2 Acciones / Tools

Tool	Agente Owner	Descripción
ingest_document	Ingestor	Parsea y clasifica un archivo subido; actualiza el índice RAG incremental
extract_topics	Ingestor	Analiza un archivo subido y extrae los temas que este contiene
retrieve_chunks	Todos	Busca fragmentos relevantes en la base vectorial dado un tema o query
generate_exam	ExamGenerator	Crea un examen con preguntas de opción múltiple y respuesta libre sobre un tema
generate_exercise	ExerciseGenerator	Genera ejercicios prácticos complejos con enunciado y datos de contexto
evaluate_answer	Evaluator	Compara respuesta del estudiante con respuesta base y el material RAG; devuelve score y feedback

Tool	Agente Owner	Descripción
ocr_math_extract	Ingestor / UI	Extrae texto y expresiones matemáticas de imágenes; genera LaTeX
update_student_profile	SupportAgent	Actualiza estadísticas, áreas débiles y preferencias del usuario en la BD
get_student_summary	SupportAgent	Recupera el perfil completo del estudiante para personalizar la generación

4. Arquitectura del Sistema

4.1 Visión General de Componentes

El sistema se organiza en tres capas principales: la capa de interfaz de usuario (UI), la capa de orquestación de agentes y la capa de datos persistentes. Los agentes se comunican a través de un bus de mensajes interno (estado compartido de LangGraph).

Loop del Agente — Estrategia ReAct + Plan-and-Execute

Cada agente sigue el patrón ReAct: emite un pensamiento interno, decide qué herramienta invocar, recibe el resultado de la herramienta, razona sobre él y decide el siguiente paso o entrega la respuesta final. El Orchestrator aplica Plan-and-Execute para tareas multi-paso (como generar un examen completo: planificar temas, recuperar chunks, generar preguntas, validar, formatear).

4.2 Agentes y Responsabilidades

Agente	Tipo de Loop	Responsabilidades Principales
Orchestrator	Plan-and-Execute	Coordina el flujo entre agentes; mantiene el estado de la sesión; decide qué agente activar en función de la intención del usuario
Ingestor Agent	ReAct	Parsea archivos, clasifica contenido (apuntes/examen/ejercicio), construye/actualiza el índice RAG de forma incremental, confirma con el usuario en casos de baja confianza en OCR
ExamGenerator	ReAct + Tools	Genera exámenes con MC y preguntas abiertas; usa <code>retrieve_chunks</code> ; respeta preferencias del usuario; proporciona respuestas base para el Evaluador
ExerciseGenerator	ReAct + Tools	Genera ejercicios prácticos complejos orientados a la resolución; puede basar los datos en contexto del material ingestado
Evaluator Agent	Chain-of-Thought	Corrige respuestas libres contrastando con la respuesta base y los chunks RAG; produce score (0-10), justificación y sugerencias de repaso
Support Agent	Reactive	Actualiza y consulta el perfil del estudiante; genera recomendaciones de foco de estudio; alimenta el dashboard de progreso

4.3 Módulo RAG

Componente RAG	Decisión de Diseño
Chunking	Chunking semántico: se divide por sección/tema detectada en el índice del documento. Fallback: chunks de 512 tokens con 64 de overlap
Embeddings	text-embedding-3-small (OpenAI) o all-MiniLM-L6-v2 (local). A definir
Vector Store	ChromaDB (local, persistente). Colecciones separadas por sesión/materia
Índice Temático	Árbol de temas extraído por el Ingestor. Permite filtrar chunks por tema antes del similarity search
Retriever	Top-K similarity search (K=5-8). Query construida dinámicamente por el agente en función del tema requerido
Actualización Incremental	Al ingerir un nuevo archivo, se agregan los nuevos chunks sin re-procesar los existentes. El índice temático se fusiona

4.4 Memoria

Tipo de Memoria	Implementación	Contenido
Corto plazo (conversacional)	Context window del LLM	Historial del turno actual: mensajes, resultados de tools, razonamiento parcial
Largo plazo (perfil de usuario)	SQLite / JSON persistente	Historial de sesiones, scores por tema, preferencias, temas con debilidad identificada
Episódica (material)	ChromaDB vectorial	Chunks del material ingestado; metadatos: tipo de documento, tema, fecha de ingesta

4.5 Stack Tecnológico

Capa	Tecnología	Justificación
Modelo LLM	Groq (modelo a definir)	Modelos gratuitos disponibles.
Orquestación	LangGraph (Python)	Soporte nativo para grafos de agentes con estado persistente; fácil de visualizar y debuggear
RAG / Vector Store	ChromaDB + LangChain	Open-source, local, sin dependencias externas; fácil integración con LangChain loaders
OCR Matemático	Mathpix API / pix2tex (local)	Estado del arte en reconocimiento de fórmulas LaTeX; Mathpix como fallback confiable
Parsing de documentos	markdown (Microsoft)	Convierte PDF/DOCX/PPTX a Markdown estructurado; preserva tablas y listas
UI Web	Next.js + React + Tailwind	SPA moderna; soporte para upload de archivos, chat, dashboard; deploy sencillo
Backend API	FastAPI (Python)	Async, tipado, OpenAPI automático; fácil integración con LangGraph y ChromaDB
Persistencia	SQLite (perfil usuario)	Sin necesidad de servidor de BD; suficiente para datos de un grupo reducido de usuarios
Observabilidad	Langfuse (open-source)	Trazas estructuradas de LLM calls + tool calls; dashboard web; LLM-as-judge integrado